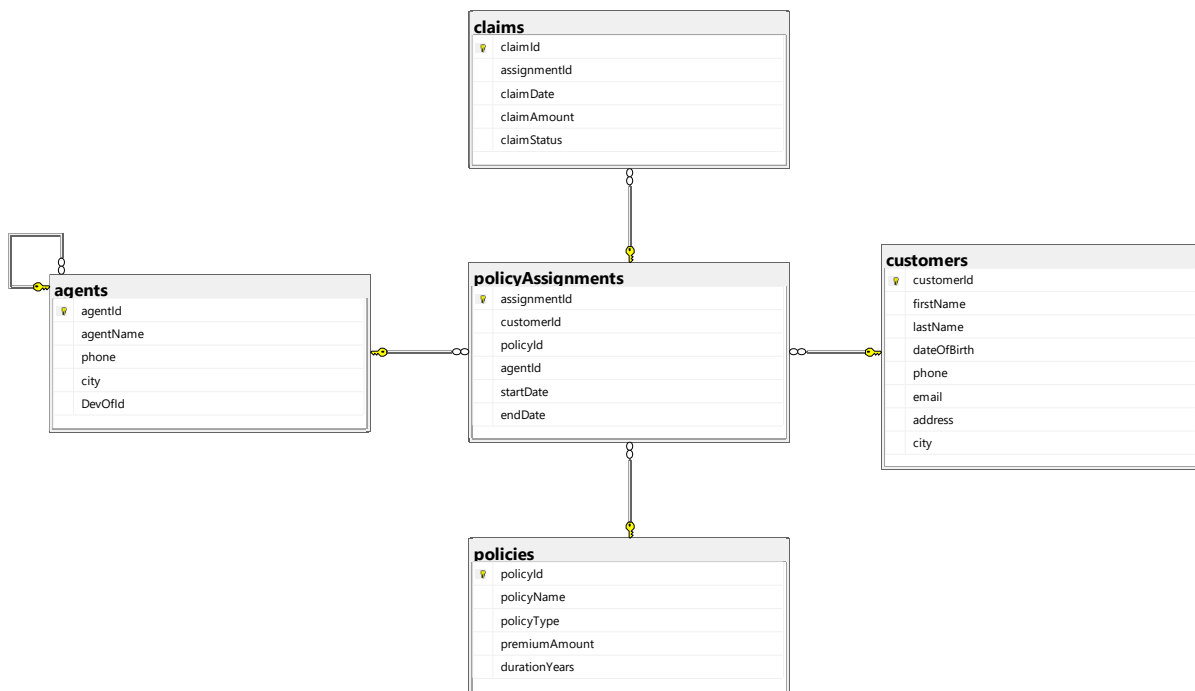


Module 4.4 Practical Project Assignment(InsuranceDB)

Schema Diagram



Creating Tables

--customers table creation

```
create table customers (  
    customerId int primary key,  
    firstName varchar(10) null,  
    lastName varchar(50) not null,  
    dateOfBirth date not null,  
    phone varchar(12) not null,  
    email varchar(50) not null  
);
```

--policies table creation

```
create table policies(  
    policyId int primary key,  
    policyName varchar(20),  
    policyType varchar(20),
```

```
premiumAmount decimal(10,2),  
durationYears int  
)
```

--claims table creation

```
create table claims (  
    claimId int primary key,  
    assignmentId int,  
    claimDate date,  
    claimAmount decimal(10,2),  
    claimStatus varchar(12),  
    foreign key (assignmentId)  
        references policyAssignments(assignmentId)  
        on delete cascade  
);
```

--agents table creation

```
create table agents(  
    agentId int primary key,  
    agentName varchar(50),  
    phone varchar(12),  
    city varchar(50)  
)
```

--policy assignments table creation

```
create table policyAssignments(  
    assignmentId int primary key,  
    customerId int,  
    policyId int,  
    agentId int,  
    startDate date,
```

```
endDate date,  
foreign key (customerId) references customers(customerId),  
foreign key (policyId) references policies(policyId),  
foreign key (agentId) references agents(agentId)  
)
```

Insertion into tables

1. Inserting data into customers

insert into customers values

```
(1,'Amit','Sharma','1995-10-10','9989727567','amit@mail.com',NULL,NULL),  
(2,'Preethi','Varma','1998-09-08','8989898989','preethi@mail.com',NULL,NULL),  
(3,'Vishwa','Nair','2014-10-10','8976598765','vishwa@mail.com',NULL,NULL),  
(4,'Hansy','Kamtala','2008-10-08','8328118765','hansika@mail.com',NULL,NULL),  
(5,'Julien','Gou','2000-09-08','9989767898','julien@mail.com',NULL,NULL);
```

2. Inserting data into agents

insert into agents values

```
(101,'Santhosh','9989898988','Hyderabad',NULL),  
(102,'Ravi','9765898756','Delhi',NULL),  
(103,'Girish','8978789789','Hyderabad',NULL),  
(104,'Hithesh','8338334564','Kakinada',NULL);
```

3. Inserting data into policies

insert into policies values

```
(120,'Health Secure','Health',17569.20,1),  
(121,'Life Shield','Life',25000.00,10),  
(122,'Motor Protect','Motor',8000.00,1);
```

4. Inserting data into policyAssignments

insert into policyAssignments values

```
(201,2,120,101,'2025-10-10','2025-10-11'),  
(202,1,120,101,'2020-10-10','2026-12-10'),
```

```
(205,5,122,103,'2025-12-12','2026-12-12'),  
(206,5,121,102,'2024-11-10','2025-12-31'),  
(207,5,120,102,'2025-11-12','2026-11-17');
```

5. Inserting data into claims

insert into claims values

```
(1000,202,'2020-10-11',10000.00,'Rejected'),  
(1001,205,'2023-11-12',20000.00,'Accepted'),  
(1002,206,'2025-12-29',15000.00,'Accepted'),  
(1003,207,'2025-12-31',40000.00,'Accepted');
```

Queries on Basic select commands including where, like, in, between

1. Basic Select

```
select 'Hi';
```

2. Selection from all tables

```
select * from customers;
```

```
select * from policies;
```

```
select * from policyAssignments;
```

```
select * from claims;
```

```
select * from agents;
```

3. Display the full name of all customers

```
select concat(firstName, lastName) as fullName  
from customers;
```

4. OFFSET–FETCH usage

```
select * from policies order by premiumAmount  
offset 1 rows fetch next 1 rows only;
```

5. Display customers whose last name ends with 'harma'

```
select * from customers where lastName like '%harma';
```

6. Display customers whose first name starts with 'A'

```
select * from customers where firstName like 'A%';
```

7. Display customers whose email starts with P or p

```
select * from customers where email like '[pP]%.com';
```

8. Display customers born after the year 1996

```
select * from customers where year(dateOfBirth) > 1996;
```

9. Display policies whose type is Life or Health

```
select * from policies where policyType in ('Life', 'Health');
```

10. Concatenate policyType with a closing bracket

```
select policyType + ')' from policies;
```

11. Invalid string-number concatenation

NOT compatible as datatypes vary

```
select policyType + 3 from policies;
```

12. Find policies with PremiumAmount BETWEEN 10,000 AND 30,000

```
select * from policies where premiumAmount between 10000 and 30000;
```

13. Retrieve claims made between '2024-01-01' and '2024-12-31'

```
select * from claims where claimDate between '2024-01-01' and '2024-12-31';
```

Operators

1. Find all customers who live in 'Delhi'

```
select * from customers where city = 'Delhi';
```

2. Display policies with PremiumAmount greater than 25,000

```
select * from policies where premiumAmount > 25000;
```

3. List claims where ClaimAmount is less than 50,000

```
select * from claims where claimAmount < 50000;
```

4. Find policies where PolicyType is 'Health' AND PremiumAmount > 20,000

```
select * from policies where policyType = 'Health' and premiumAmount > 20000;
```

5. Display claims where ClaimStatus is NOT 'Approved'

```
select * from claims where claimStatus <> 'Approved';
```

String functions

ASCII and CHAR

```
select ascii('A');
```

```
select char(65);
```

CHARINDEX

```
select charindex('honey', 'aaaahoneycomb');
```

```
select charindex('honey', 'aaaahoneycombhoney', 6);
```

CONCAT + LEFT

```
select concat(left(policyType, 5), policyId) from policies;
```

REPLICATE

```
select replicate('abhi', 3);
```

LEN

```
select len('SQLServer');
```

String functions On Insurance Table

1. Display customer full name by concatenating FirstName and LastName

```
select concat(firstName, lastName) as fullName from customers;
```

2. Find customers whose FirstName starts with 'A'

```
select * from customers where firstName like 'A%';
```

3. Display Policy Type and its character length

```
select policyType, len(policyType) from policies;
```

4. Extract last 4 characters from PolicyType

```
select right(policyType, 4) from policies;
```

5. Find customers whose LastName length is more than 6 characters

```
select * from customers where len(lastName) > 6;
```

6. Replace the word 'Life' with 'Term Life' in Policy Type

```
select replace(policyType, 'Life', 'Term Life') from policies;
```

Numeric Functions

1. PremiumAmount rounded to 2 decimal places

select round(premiumAmount, 2) from policies;

2. Display Policy Names where PremiumAmount is greater than 10,000

select policyName from policies where premiumAmount > 10000;

3. Display PremiumAmount with CEIL function

select ceiling(premiumAmount) from policies;

4. Display Policy Name and total Premium including 18% GST

select policyName, premiumAmount * 1.18 as totalPremium from policies;

5. Calculate 10% of PremiumAmount

select premiumAmount * 0.1 from policies;

DATE functions

1. Display current date

select getdate();

2. Display only current date

select cast(getdate() as date);

3. Display current time

select cast(getdate() as time);

4. Display year, month and day separately

select datepart(year, getdate()) as year, datepart(month, getdate()) as month,
datepart(day, getdate()) as day;

5. Display policy assignments started in the year 2020

select * from policyAssignments where datepart(year, startDate) = 2020;

6. Display policy assignments that started in October

select * from policyAssignments where month(startDate) = 10;

7. Display policy assignments that started on a Monday

select * from policyAssignments where datepart(weekday, startDate) = 2;

8. Display policy assignments whose policy duration is more than 1 year

select *, datediff(year, startDate, endDate) as durationInYears from policyAssignments
where datediff(year, startDate, endDate) > 1;

9. Display number of days remaining for each policy

select assignmentId, startDate, endDate, datediff(day, getdate(), endDate) as daysRemaining from policyAssignments;

10. Display ClaimDate in DD-MM-YYYY format

select claimDate, format(claimDate, 'dd-MM-yyyy') as formattedClaimDate from claims;

11. customer age from Date of Birth (DOB)

select customerId, firstName, lastName, datediff(year, dateOfBirth, getdate()) as age from customers;

Aggregate Functions

1. Display policies ordered by duration

select * from policies order by durationYears;

2. Display policies ordered by duration and premium

select * from policies order by durationYears, premiumAmount;

3. Display number of people assigned to each policy

select policyId, count(*) as noOfPeopleWithPolicy from policyAssignments
group by policyId;

4. find total claimAmount per claimStatus

select claimStatus, sum(claimAmount) from claims group by claimStatus;

5. Find Policy Types where average premium > 6000.

select policyType from policies group by policyType having avg(premiumAmount)>10000 ;

Joins

1. List all customers with their policy types.

select firstName, policyType from customers join policyAssignments on
policyAssignments.customerId=customers.customerId join policies on
policyAssignments.policyId=policies.policyId;

2. Find customers who have no policy.

select firstName from customers left join policyAssignments on
policyAssignments.customerId=customers.customerId where policyAssignments.policyId
is null;

3. Display policy details along with claim amount (if any).

```
select p.*,c.claimAmount from policies p join policyAssignments pa on  
pa.policyId=p.policyId join claims c on c.assignmentId=pa.assignmentId;
```

4. Show customers with more than one policy.

```
select distinct concat(firstName, ' ',lastName) from customers join policyAssignments on  
policyAssignments.customerId=customers.customerId;
```

5. Show policy count per policy type.

```
select p.policyName,count(customerId) as policyCount from policyAssignments join  
policies p on p.policyId=policyAssignments.policyId group by  
policyAssignments.policyId,p.policyName;
```

Subqueries

1. Find customers who have taken premium above average

```
select distinct c.customerId, c.firstName, c.lastName from customers c  
join policyAssignments pa on c.customerId = pa.customerId  
join policies p on pa.policyId = p.policyId  
where p.premiumAmount > (select avg(premiumAmount) from policies);
```

2. Display policies having the highest premium

```
select * from policies where premiumAmount = (select max(premiumAmount) from  
policies);
```

3. Find customers with no claims

```
select * from customers where customerId not in  
( select pa.customerId from policyAssignments pa join claims c on pa.assignmentId =  
c.assignmentId);
```

4. Policies taken by more than one customer

```
select * from policies where policyId in  
( select policyId from policyAssignments group by policyId having count(distinct  
customerId)> 1);
```

5. Policies with no assignments

```
select * from policies where policyId not in  
(select policyId from policyAssignments);
```

Set Operators

1. Customers who bought Life OR Health policies.

```
select c.* from policyAssignments pa
join policies p on pa.policyId = p.policyId
join customers c on c.customerId=pa.customerId
where p.policyType = 'Life'

union

select c.*
from policyAssignments pa
join policies p on pa.policyId = p.policyId
join customers c on c.customerId=pa.customerId
where p.policyType = 'Health';
```

2. Customers who bought both Life AND Health.

```
select c.* from policyAssignments pa
join policies p on pa.policyId = p.policyId
join customers c on c.customerId=pa.customerId
where p.policyType = 'Life'

intersect

select c.* from policyAssignments pa
join policies p on pa.policyId = p.policyId
join customers c on c.customerId=pa.customerId
where p.policyType = 'Health';
```

3. Customers who bought Life but NOT Health.

```
select c.* from policyAssignments pa
join policies p on pa.policyId = p.policyId
join customers c on c.customerId=pa.customerId
where p.policyType = 'Life'

except
```

```
select c.* from policyAssignments pa
join policies p on pa.policyId = p.policyId
join customers c on c.customerId=pa.customerId
where p.policyType = 'Health';
```

4. All customers with policies (without duplicates).

```
select c.* from policyAssignments join customers c on c.customerId=pa.customerId
union
select c.* from policyAssignments join customers c on c.customerId=pa.customerId;
```

5. Policies having claims OR assignments.

```
select policyId from policyAssignments
union
select pa.policyId from claims c join policyAssignments pa on c.assignmentId =
pa.assignmentId;
```

6. Common policies in Claim and Assignment.

```
select policyId from policyAssignments
intersect
select pa.policyId from claims c join policyAssignments pa on c.assignmentId =
pa.assignmentId;
```

MERGE

merging new customer data from source list into main customers table (updates when matched)

```
merge customers as target
using (
    select 1 as customerId, 'Amit' as firstName, 'Sharma' as lastName, 'Delhi' as city
    union all
    select 6, 'Rahul', 'Verma', 'Mumbai'
) as source
on target.customerId = source.customerId
```

when matched then

update set

target.firstName = source.firstName,

target.lastName = source.lastName,

target.city = source.city

ROLLUP

To display total premium collected by policyType and overall total

```
select p.policyType, sum(p.premiumAmount) as totalPremium from policies p group by  
rollup(p.policyType);
```

CUBE

To get aggregate claimAmounts categorized based on claimStatus and year of claim

```
select claimStatus, year(claimDate) as claimYear, sum(claimAmount) as totalClaimAmount  
from claims group by cube (claimStatus, year(claimDate));
```

GROUPING

To display total premium by Policy Type and Grand Total using ROLLUP and identify total rows by customizing nulls with grouping

```
select case when grouping(p.policyType) = 1 then 'Grand Total' else p.policyType end as  
policyType, sum(p.premiumAmount) as totalPremium from policies p group by  
rollup(p.policyType);
```